

Extração de informações com Gemini

Relatório técnico do desafio Ir Além 1

Felipe Livino dos Santos | RM 563187; Daniel Veiga Rodrigues de Faria | RM 561410; Tomas Haru Sakugawa Becker | RM 564147; Daniel Tavares de Lima Freitas | RM 562625; Gabriel Konno Carrozza | RM 564468.

1 Proposta e relação com o assistente

O primeiro desafio adicional amplia o Cardiolo para trabalhar com informações que chegam em texto livre. Uma pessoa pode mencionar uma medição, dizer que não apresenta determinado sintoma e comentar quando algo começou, tudo na mesma mensagem. A leitura desse conteúdo exige separar os fatos sem perder a maneira como foram declarados. Por isso, a extensão produz uma estrutura que pode ser conferida pelo usuário antes de ser incorporada à conversa.

Optamos pela extração de textos fictícios, uma das alternativas previstas no enunciado. Não foi implementada análise de imagens. A solução usa o Gemini por meio da biblioteca google-genai e do modelo configurado em GEMINI_MODEL, cujo valor padrão é gemini-3.5-flash-lite. O uso de uma API de modelo comercial e de instruções de extração se relaciona aos exemplos de integração apresentados no capítulo 10 de PCV, especialmente nas páginas 57 a 63 [1].

A extração complementa o Watson, que continua responsável por conduzir o diálogo. Essa separação evita que uma geração livre passe a decidir, por conta própria, qual etapa do atendimento executar. O serviço generativo recebe um texto e devolve fatos dentro de um contrato definido; a interface apresenta esses fatos e a pessoa escolhe se deseja incluí-los no contexto da conversa.

2 Percurso do texto pela aplicação

Na aba Organizar relato, o usuário escreve ou cola um texto e solicita a organização. A requisição chega ao Flask, que verifica o formato JSON, o token de proteção e o limite de 6.000 caracteres do relato. O serviço de extração verifica se existe uma resposta recente para a mesma combinação de modelo, prompt e texto. Quando há um resultado válido em cache, ele é reutilizado sem uma nova chamada ao provedor.

Quando não há resultado em cache, a aplicação reserva uma tentativa no controle local de uso e envia o texto ao Gemini. A chamada inclui uma instrução de sistema, o esquema de resposta esperado e a solicitação de saída em JSON. Depois da geração, o backend valida a estrutura e confere os trechos de evidência. Apenas uma resposta que passa por essas verificações é disponibilizada para revisão e armazenada em cache.

O fluxo pode ser acompanhado em sete momentos: entrada do relato, verificação da requisição, consulta ao cache, reserva de tentativa, geração, validação e revisão. Essa sequência ajuda a localizar falhas. Um erro de autenticação não é apresentado como ausência de fatos, e uma saída inválida não é silenciosamente transformada em um resumo aparentemente correto.

3 Instruções e contrato da resposta

O prompt pede que o modelo extraia somente informações declaradas de forma explícita. Também orienta a preservar maiúsculas, unidades, acentos, negações e incertezas, sem preencher lacunas. O relato é tratado como conteúdo a analisar, não como uma nova instrução para o assistente. Se o texto contiver um pedido para inventar uma medição, essa solicitação não deve alterar o contrato de extração.

A saída principal contém uma lista chamada fatos. Cada elemento precisa informar o campo, o valor, a evidência literal e o estado da declaração. Os campos aceitos são sintoma, pressao_arterial, frequencia_cardiaca, inicio, historico e adesao. O estado pode ser afirmado, negado ou incerto. Um texto sem informações dessas categorias deve produzir uma lista vazia, em vez de um preenchimento por aproximação.

Elemento	Função no resultado
campo	Identifica a categoria prevista no esquema, como frequência cardíaca ou histórico.
valor	Conserva o trecho que representa a informação, sem inventar unidade ou converter a medida.
evidencia	Mostra a passagem do texto que sustenta o valor e permite conferir a declaração.
estado	Distingue informação afirmada, negada ou incerta.

A implementação usa Pydantic em modo estrito e rejeita propriedades desconhecidas. A lista admite até 30 fatos; valor e evidência precisam ser textos não vazios, com limites próprios de tamanho. Além disso, a evidência deve aparecer literalmente no relato, e o valor deve estar contido nessa evidência. Assim, não basta que o modelo entregue um JSON bem formado: ele também precisa indicar de onde retirou cada informação.

4 Exemplo de interpretação

Considere o relato fictício “Minha frequência foi 72 bpm. Não tenho dor.”. Uma saída compatível separa a frequência de 72 bpm, com estado afirmado, e o valor “dor”, com estado negado e evidência “Não tenho dor.”. A presença da palavra dor, isoladamente, não permite registrar o sintoma como presente. O trecho de origem é o que dá sentido ao valor.

```
{"campo": "sintoma", "valor": "dor",  
  "evidencia": "Não tenho dor.", "estado": "negado"}
```

Esse fragmento ilustra um fato válido dentro da lista. Se duas medidas forem contraditórias ou o relato demonstrar dúvida, a instrução exige preservar a incerteza. A aplicação não deve escolher qual valor está correto. Da mesma forma, “12/8” não é convertido automaticamente para “120/80”, pois essa transformação acrescentaria uma interpretação que o usuário não confirmou.

5 Revisão humana e integração com a conversa

A interface apresenta o valor extraído, sua categoria, o estado e a passagem que serviu de evidência. Essa disposição permite revisar o resultado sem começar pela leitura do JSON completo, que continua disponível em uma área expansível. O botão de inclusão aparece quando existe uma conversa ativa e há fatos a revisar. A extração também pode ser usada isoladamente, sem abrir uma sessão de diálogo.

Ao incluir o relato, o backend envia o texto original e os fatos revisados ao contexto do Watson e solicita a conferência do resumo. A ação não transforma todos os fatos gerados em medições do painel. Os valores numéricos estruturados da conversa continuam sujeitos ao fluxo próprio de registro. Essa escolha evita que uma inferência do modelo substitua um dado confirmado apenas por ter sido produzida em formato JSON.

A confirmação é uma conferência do conteúdo pelo usuário. Ela não valida um diagnóstico nem comprova a correção clínica de uma informação. A revisão deve observar, principalmente, se há omissões, se a evidência contém a negação completa, se uma dúvida foi preservada e se o modelo selecionou um trecho que realmente corresponde ao campo indicado.

6 Controle de chamadas e tratamento de falhas

O aplicativo e o robô compartilham um controle local persistente de até 500 tentativas em uma janela móvel de 24 horas. O intervalo mínimo padrão entre tentativas é de dez segundos. A reserva ocorre em uma transação SQLite, para que duas requisições concorrentes não consumam a mesma posição disponível. Uma falha também conta como tentativa, e não há repetição automática da chamada.

O cache permanece válido por 24 horas e usa uma chave derivada do modelo, do prompt e do texto. Alterar uma dessas partes muda a chave, evitando reutilizar uma resposta associada a instruções diferentes. A persistência no volume Docker conserva esse controle após a recriação dos containers. O número mostrado na aplicação corresponde ao uso local; ele não representa toda a utilização da conta ou do projeto no Google.

A ausência de chave, a indisponibilidade do modelo, uma limitação de chamadas ou uma falha na validação produzem mensagens distintas. Nenhum desses casos deve resultar em fatos fabricados para manter a tela preenchida. Há ainda uma triagem conservadora que rejeita fatos afirmados quando a evidência contém palavras como “não” ou “sem”. Essa regra é simples: pode rejeitar uma construção válida e também não alcança todas as formas de negação. Por isso, ela não substitui a revisão do texto.

7 Testes e resultados documentados

Os testes automatizados verificam o contrato da extração, a presença literal das evidências, a conservação de negações, a rejeição de chaves não previstas e o comportamento quando falta configuração. Também exercitam o cache, a persistência e a concorrência no orçamento de chamadas. Nesses testes, as respostas externas são controladas. Eles ajudam a detectar regressões do código, mas não medem a qualidade de compreensão do modelo.

A avaliação com a API real registrada em 4 de setembro de 2026 reuniu seis casos fictícios: medidas com negação, ausência de medição, duas medidas, texto sem fatos previstos, instrução inserida para inventar um valor e incerteza. As seis respostas passaram pelas verificações de esquema e evidência; os seis fatos previstos na referência inicial foram encontrados. O primeiro caso reutilizou o cache de uma chamada real anterior, enquanto os outros cinco envolveram novas chamadas [2].

No caso de incerteza, o modelo também devolveu uma informação de início, “desde ontem”, sustentada pelo relato e ausente da referência inicial. A revisão considerou o fato válido e manteve o registro da divergência. Esse episódio mostra que a própria referência precisa ser examinada: uma diferença entre resposta e gabarito não significa, necessariamente, que o modelo inventou um dado.

Não foram observados fatos sem apoio nessas seis respostas. A amostra, porém, é pequena e não comprova resistência geral a instruções maliciosas, ausência de alucinações ou desempenho clínico. A variação entre gerações independentes não foi medida. A revisão posterior da conversa não refez essa avaliação do Gemini; os resultados apresentados aqui pertencem à execução documentada, e devem ser lidos com essa delimitação.

8 Reprodução e conclusão

Para reproduzir o fluxo, é necessário preencher `GEMINI_API_KEY` no ambiente local, configurar o modelo e recriar os containers para aplicar as variáveis. A extração pode ser feita pela interface ou pelo módulo `extensions.generative.extract_clinical`. O módulo de avaliação executa casos de demonstração e grava evidências no volume. O README descreve os comandos e os cuidados para evitar disputa de intervalo entre aplicação e robô.

A contribuição desta extensão está em tornar o relato revisável: cada informação fica associada ao trecho que a sustenta e ao estado em que foi declarada. O resultado é adequado à demonstração acadêmica de prompting, integração por API e saída estruturada. Uma evolução do projeto deve ampliar a variedade de relatos e avaliar omissões, ambiguidades e repetibilidade antes de propor qualquer uso fora do ambiente simulado.

[1] FIAP. Arquitetura Cognitiva dos LLMs Modernos. PCV, Fase 5, capítulo 10, p. 57 a 63. Material didático da atividade.

[2] Projeto Cardiola. docs/evidence/gemini-evaluation.json e docs/VALIDACAO.md. Avaliação registrada em 4 set. 2026. Implementação consultada: services/gemini_service.py, services/storage.py e extensions/generative/prompt.txt.